

Programación Colaborativa: git y GitHub

Universidad Rey Juan Carlos

- 1 Desarrollo colaborativo
- 2 Conceptos básicos
- 3 Sistemas de Control de Versiones
- 4 Políticas y Metodologías
- 5 Git
- 6 Ejercicios

Introducción

- La colaboración en equipo es cada día más común , y se utilizan diversos mecanismos y modelos de trabajo:
 - Versionado: documento.docx, documento v2.docx, ...
 - Herramientas: Dropbox, GDrive, adjuntos por correo, ...
- Sin embargo, estas metodologías no escalan bien en proyectos de desarrollo.
- Para ayudar en esta tarea existen los **sistemas de control de versiones** (SCV).
- Un SVC permite:
 - Crear copias de seguridad y restaurarlas.
 - Sincronizar a los desarrolladores respecto de la última versión de desarrollo.
 - Deshacer cambios, tanto puntuales como historicos.
 - Gestionar la auditoría del código: quién hizo qué y cuándo.
 - Realizar pruebas aisladas a través del mecanismo de ramas.



Terminología habitual en SCV (I)

Repositorio

- Almacén que guarda toda la información del proyecto.
- Suele tener estructura de árbol.

Servidor

- Máquina donde está alojado el repositorio.

Working Copy/Working Set (Copia de trabajo)

- Copia local donde el desarrollador trabaja.

Trunk/Main/master (Rama principal)

- Localización dentro del repositorio.
- Contiene la rama principal de desarrollo.

SCM

- Sigla de *Source* Code Management, o gestión de código fuente. Propiedad aplicable a los SCV.

Terminología habitual en SCV (II)

Operaciones básicas

- **Add**: añade un archivo para que sea rastreado por el SCV.
- **Revisión**: versión de un archivo/directorio dentro del SCV.
- **Head**: última versión del repositorio (completo o de una rama).
- **Check out**: creación de una copia de trabajo que rastrea un repositorio.
- **Check in / commits**: envíe cambios locales al repositorio. Como resultado cambia la versión del archivo(s)/repositorio.
- **Mensaje de Check in/log**: todo *check in* tiene asociado un mensaje que describe la finalidad del cambio. Puede estar asociado a un sistema de gestión de incidencias (*tickets*).
- **Log (Historia)**: permite visualizar/revisar la lista de cambios de un archivo/repositorio.
- **Update/Synchronize/fetch&pull** (Actualizar): sincroniza la copia de trabajo con la última versión que existe en el repositorio.
- **Revert/Reset** (Deshacer): permite deshacer los cambios realizados en la copia de trabajo y dejar el archivo/recurso en el último estado conocido del repositorio.

Terminología habitual en SCV (III)

Operaciones avanzadas Branching (ramas)

- Permite crear una copia de un archivo/carpeta rastreada.
- Permite desarrollar en paralelo en otra “rama” pero dejando constancia de la relación que existe con la rama original.

Diff/Change/Delta (Cambio)

- Permite encontrar las diferencias entre dos versiones del repositorio.
- Se puede generar un parche que permitiría pasar de una versión a otra.

Merge/Patch

- Aplica los cambios de un archivo a otro. Utilizado
- habitualmente para mezclar branches.

Conflict (Conflicto)

- Problema que surge cuando varios desarrolladores modifican el mismo recurso y los cambios se solapan.

Tipos de Sistemas de Control de Versiones

Centralizados

- Existe un servidor centralizado que almacena el repositorio completo.
- La comunicación/colaboración entre desarrolladores se lleva a cabo (forzosamente) a través del repositorio centralizado.
- Son más sencillos de utilizar.
- Los modelos de trabajo son más restringidos.

Distribuidos

- Cada desarrollador contiene una copia completa de todo el repositorio.
- Los mecanismos de comunicación/colaboración entre desarrolladores son más abiertos.
- Son (un poco) más difíciles de utilizar que los sistemas centralizados. Los
- modelos de trabajo son más flexibles.
- “Los branches/merges son más simples”.

Herramientas SCV

Centralizados

- **Subversion (SVN)**
 - <http://subversion.apache.org>
 - <http://subversion.tigris.org>
- Concurrent Version System (CVS)
 - <http://www.nongnu.org/cvs/>
- Microsoft Visual Source Safe
- Perforce

Distribuidos

- **Git** - <http://git-scm.com>
- Mercurial - <https://www.mercurial-scm.org/>
- Bazaar - <http://bazaar.canonical.com/en/> DARCS -
- <http://darcs.net/>

Políticas de Control de Versiones

- Para aprovechar los SCV es necesario:
 - Establecer una política para el control de versiones en los proyectos:
 - Estructura del repositorio. Política para la rama principal.
 - ...
 - Documentar el desarrollo.
 - Utilizando alguna herramienta de gestión de seguimiento como, por ejemplo, Trac
 - <https://trac.edgewall.org/>
- Es importante adoptar un modelo de trabajo que sea adecuado para el equipo de desarrollo.
 - El modelo de trabajo influye en la herramienta SCV a utilizar.

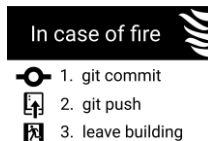
Metodología Básica de Trabajo

La primera vez:

- 1 Creación del repositorio del proyecto (opcional).
 - Importación inicial de código del proyecto (opcional).
- 2 Creación de una copia de trabajo del repositorio.
- 3 Modificación de la copia de trabajo del repositorio.
- 4 Envíe cambios al repositorio.

Siguientes ocasiones:

- 1 Actualizar el repositorio.
- 2 Modificar la copia de trabajo.
- 3 Envíe cambios al repositorio.

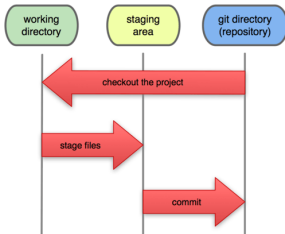


Ventajas de git

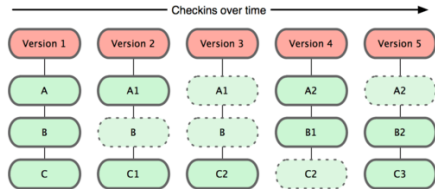
- Branches locales muy útiles y “baratos”:
 - Fáciles de crear y borrar.
 - No tienen por qué ser públicos.
 - Útiles para organizar el trabajo y los experimentos.
- Las operaciones en git se gestionan en local:
 - Las operaciones son más rápidas.
 - Se puede trabajar sin acceso a la red.
 - Todos los repositorios de los desarrolladores son iguales, así que en caso de emergencia, pueden servir de backup.
- El espacio ocupado no es grande, pese a guardar una copia de todo el repositorio:
 - Si dos archivos son iguales solamente se guarda el contenido de uno. El contenido de los archivos se guarda comprimido.
 - Se compactan los archivos periódicamente, generando *deltas* entre diferentes versiones de los archivos.
- Git tiene integridad: todo (archivos, carpetas, etc.) tiene una firma SHA1 asociada.

Ventajas de git (II)

Local Operations



Versionado de ficheros



- La *staging area* (SA) ~~área~~ **área de ensayo** (también llamada índice) es la zona donde se añaden los cambios que se van a hacer en el commit.
 - No es necesario añadir todos los archivos de la WC a la SA.
 - Es posible añadir a la SA modificaciones parciales de archivos (*hunks*). De este modo, cada modificación parcial puede ir en un commit.
- Buena práctica en git: **hacer commits frecuentes**.
 - Pequeños y que incluyan solamente las modificaciones concretas que resuelvan un problema o tarea.

Instalación de git

Básico:

- Linux: `sudo apt-get/yum install git (git-cola, git-meld)`
- Mac OSX: hay que activar XCode; git viene integrado.
<https://developer.apple.com/xcode/>
- Windows: <https://git-scm.com/download/win>, o bien instalar Cygwin + git.

Entornos gráficos:

- SourceTree (Windows / MacOSX) (Gratuita),
<https://www.sourcetreeapp.com/>
- Smartgit (Multiplataforma) (free non-commercial),
<http://www.syntevo.com/smartgit/>
- GitX (MacOSX), <http://gitx.frim.nl/>
- GitHub Desktop (Mutiplataforma): para repositorios de GitHub,
<https://desktop.github.com/>

Instalación de git (II)

La mayoría de los IDE integran alguna vista para gestión de repositorios:

- Netbeans
- Eclipse
- IntelliJ

Después de instalar git suelen quedar algunas cuestiones de post-instalación:

- Configurar el editor de mensajes de commit.
- Configurar el gestor de conflictos (mergetool).

- Servicio de git gestionado.
- Versión gratuita, con repositorios públicos y ahora también privados:
<https://github.com/>
- Versión “Education”:
 - <https://education.github.com/>
 - 5 repos privados.
 - Acceso a descuentos en herramientas colaborativas.
- Versión de pago con diferentes características.



Ejercicio guiado

Construir un repositorio

- Crea una cuenta en github. Crea un repositorio. Identifica la URL de tu repositorio (la necesitarás después). Invita a colaboradores (tus compañeros de grupo).
- Instala Git.
- Crea un programa “HolaMundo” en IntelliJ/NetBeans.
- Abre el bash de GIT.
- Ubícate en el workspace de tu HolaMundo
- Introduce esta secuencia de comandos:
 - `git init`
 - `git add .`
 - `git commit -m “Primer commit”`
 - `git branch -M main`
 - `git remote add origin https://github.com/USUARIO/PROYECTO.git`
 - `git push -u origin main` **--OJO, este paso te puede pedir confirmación--**

Ejercicio guiado

Contribuir a un repositorio

- Crea una cuenta en github. Acepta la invitación de colaborar de tu compañero.
- Instala Git.
- Con Git:
 - Abre el bash de GIT.
 - Ubícate en el workspace donde quieres que esté tu proyecto
 - Introduce esta secuencia de comandos:
 - `git clone https://github.com/USUARIO/PROYECTO`
- Con IntelliJ:
 - Crea un nuevo proyecto con la opción File->New->Project From Version Control
 - Puedes poner directamente la URL o incluso hacer login desde ahí con GitHub y te ofrece clonar los repositorios tuyos o donde colaboras.

- Con NetBeans:
 - Crea un nuevo proyecto con la opción Team->Git->Clone...
 - Puedes poner directamente la URL y vuestro usuario y password de github.

Nota: Es posible que el password de github no os funcione para conectaros en NetBeans y tengáis que crear un token con el scope “repo”. (Los tokens se crean desde las settings de vuestra cuenta de github, en “developer settings” y “generar new token”).

Ejercicios propuestos

- 1 Cread ramas distintas para añadir distintos tipos de saludo “Hola Mundo”.
- 2 Combinad todo el trabajo en la rama máster.
- 3 Cread un conflicto creando un método de saludo en una rama y en otra el mismo método (mismo nombre) pero que haga otra cosa. Haced merge y solucionad el conflicto.

Referencias

- Libro de git “oficial”: <https://git-scm.com/book/es/v1>
 - Aprende git: <http://aprendegit.com/>
 - Git core tutorial: <https://www.kernel.org/pub/software/scm/git/docs/gitcore-tutorial.html>
 - Ayuda GitHub: <https://help.github.com/>
<https://help.github.com/articles/good-resources-for-learning-git-and-github/>
- Chuleta Git:
- <https://education.github.com/git-cheat-sheet-education.pdf>